

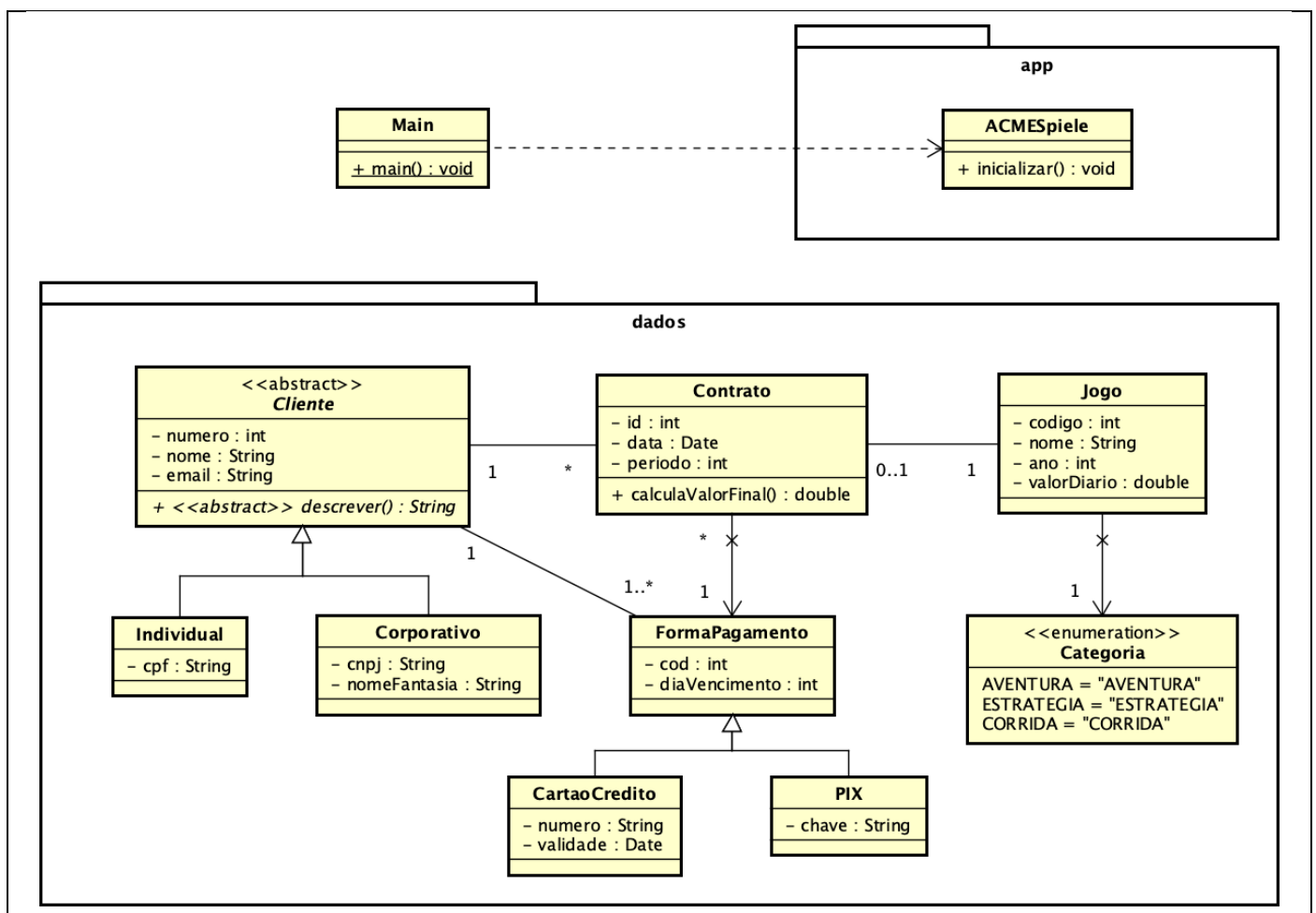
Trabalho II

Contrato de jogos

1. Enunciado geral:

A ACMEWebbasierteVideospiele é uma empresa que gerencia os contratos e uso de jogos eletrônicos *online* e que deseja automatizar seu negócio.

O analista de sistemas gerou um diagrama de classes inicial, com alguns atributos, operações e relacionamentos apresentados a seguir.



O método **inicializar()** da classe **ACMESpiele** deve ler dados de arquivos de entrada para cadastrar alguns clientes, jogos e contratos iniciais (os formatos dos arquivos de entrada são apresentados no Apêndice deste trabalho). Cada contrato lido do arquivo de contratos deve ser armazenado em uma **fila**. Após a leitura de todos os contratos do arquivo, cada contrato válido deve ser cadastrado no sistema, sendo que o cálculo final de cada contrato deve ser realizado conforme a descrição a seguir.

O método **calculaValorFinal()** da classe **Contrato** calcula o valor final do contrato depende da categoria do jogo e da forma de pagamento:

- Conforme a categoria do jogo:
 - **AVENTURA**: adiciona 5% ao valor diário do jogo.

- **ESTRATEGIA**: adiciona 10% ao valor diário do jogo.
- **CORRIDA**: adiciona 15% ao valor diário do jogo.
- Conforme a forma de pagamento do contrato:
 - **Cartão de crédito**: adiciona 5% ao valor diário do jogo.
 - **PIX**: diminui 5% do valor diário do jogo.

Além disso, se o cliente possuir mais de 2 contratos, o valor final de cada contrato possui 3% de desconto.

Todo cliente precisa se cadastrar com seus dados; e pode ser individual (pessoa física) ou corporativo (pessoa jurídica). O método **descrever()** deve gerar uma String com os valores dos atributos separados por (;) ponto-e-vírgula.

A empresa disponibiliza diversos jogos eletrônicos exclusivos (podem ser contratados apenas por um cliente de cada vez) para serem contratados para serem jogados por um determinado período (em dias) a partir da data do contrato.

No contrato é definida uma forma de pagamento.

Além das classes iniciais, devem ser implementadas classes “catálogo” que gerenciem as coleções respectivas de Clientes, Jogos, Contratos.

A classe ACMESpiele deve apresentar uma tela cíclica com o usuário para as seguintes opções:

1. **Cadastrar cliente**: solicita todos os dados um cliente e, se o número não for repetido no sistema, cadastra-o no sistema. Se o número for repetido mostra uma mensagem de erro. Os clientes são mantidos em ordem crescente de número.
2. **Cadastrar jogo**: solicita todos os dados um jogo, mostra as categorias cadastradas para a seleção de uma categoria e se o código (codigo) não for repetido no sistema, cadastra o jogo no sistema. Se o código for repetido mostra uma mensagem de erro. Os jogos são mantidos em ordem crescente de código.
3. **Cadastrar forma de pagamento**: mostra os dados de clientes cadastrados para a seleção de um cliente. Solicita todos os dados uma forma de pagamento e, se o código (cod) não for repetido no sistema, cadastra-a no sistema. Se o código for repetido mostra uma mensagem de erro. As formas de pagamento são mantidas em ordem crescente de código.
4. **Cadastrar contrato**: mostra os dados de clientes cadastrados para a seleção de um cliente. Mostra os dados de jogos cadastrados para a seleção de um jogo. Mostra as formas de pagamento do cliente para a seleção da forma de pagamento. Solicita os dados do contrato. Se o identificador (id) não for repetido e o jogo disponível para contrato, cadastra o contrato no sistema. Se o identificador for repetido, ou se o jogo está indisponível para contratar, mostra uma mensagem de erro. Os contratos são mantidos em ordem decrescente de identificador.
5. **Mostrar relatório de jogos**: mostra todos os valores de todos os atributos dos jogos cadastrados no sistema (inclusive os dados da categoria correspondente). Se não há jogo cadastrado, mostra uma mensagem de erro.
6. **Mostrar relatório de clientes**: mostra todos os valores de todos os atributos dos clientes cadastrados no sistema, inclusive das formas de pagamento de cada cliente. Se não há cliente cadastrado, mostra uma mensagem de erro.

7. **Mostrar relatório de contratos:** mostra todos os valores de todos os atributos dos contratos cadastrados no sistema (inclusive os dados do cliente, do jogo e da forma de pagamento correspondentes e o valor final do contrato). Se não há contrato cadastrado, mostra uma mensagem de erro.
8. **Remover os dados de um determinado contrato:** mostra os dados dos contratos cadastrados para a seleção de um contrato. O contrato selecionado deve ser removido do sistema. Se não existir contratos cadastrados, mostra uma mensagem de erro.
9. **Alterar os dados de um determinado cliente:** mostra os dados dos clientes cadastrados para a seleção de um cliente. Para o cliente selecionado, solicita as novas informações do cliente (exceto o número) e atualiza-o no sistema. Se não existir clientes cadastrados, mostra uma mensagem de erro.
10. **Consultar maior:** solicita uma opção e apresenta o resultado da consulta:
 - **Consultar jogo com maior valor diário:** mostra os dados do jogo com maior valor diário cadastrado. Se houver empate, mostra todos. Se não há jogos cadastrados, mostra uma mensagem de erro.
 - **Consultar contrato com maior valor final:** mostra todos os dados do contrato com maior valor final (inclusive os dados do cliente, do jogo e forma de pagamento). Se houver empate, mostra todos. Se não há contratos cadastrados, mostra uma mensagem de erro.
 - **Consultar cliente com maior montante de contratos:** mostra os dados do cliente com maior montante (somatório de valores de seus contratos) do sistema. Se houver empate, mostra todos. Se não clientes cadastrados, mostra uma mensagem de erro.
11. **Salvar dados:** solicita ao usuário que digite, apenas uma vez, nome de arquivo desejado, sem extensão (por exemplo: GRAVADO) e salva todos os dados cadastrados no sistema em um ou mais arquivos. Se houver algum problema no salvamento mostra uma mensagem de erro.
12. **Carregar dados:** solicita ao usuário que digite, apenas uma vez, o nome de arquivo, sem extensão (por exemplo: GRAVADO) e carrega para o sistema todos os dados de um ou mais arquivos salvos anteriormente. Se houver algum problema no carregamento mostra uma mensagem de erro.
13. **Finalizar sistema:** termina a execução do sistema.

2. Definição do trabalho:

O objetivo do trabalho é implementar um sistema de gerenciamento de clientes, jogos, formas de pagamento e contratos, capaz de atender as necessidades da empresa descritas no enunciado geral, e que atenda as restrições que seguem:

- É permitida a criação de novas classes, métodos, atributos e relacionamentos, mas as informações definidas no diagrama de classes original não podem ser removidas.
- Deve haver tratamento de exceções para que não haja interrupção da execução ou mensagens de erro da JVM.
- Deve utilizar coleções da API Java (exceto array []) para o armazenamento de objetos do sistema.
- Todos os atributos devem ser privados (*private*).
- Atributos ou métodos de classe (*static*) devem ser evitados; exceções devem ser relatadas no relatório do projeto.

- Deve possuir interface gráfica para interação com o usuário.
- Deve ser entregue um relatório do projeto contendo os seguintes capítulos:
 1. Diagrama de classes – diagrama atualizado das classes do sistema ou indicação do nome do arquivo que possui o diagrama de classes;
 2. Coleções de dados – descrição de coleções (listas, filas, pilhas, etc.) que foram utilizadas e em que partes do sistema;
 3. Armazenamento (persistência) de dados – descrição da forma escolhida de armazenamento de dados em arquivos texto (TXT, CSV, JSON, XML, etc.) ou serialização.

3. Critérios de avaliação:

- O trabalho será avaliado conforme os seguintes critérios:
 - Nota do Trabalho = 10% apresentação + 90% implementação.
 - A implementação envolve:
 - Relatório do projeto: 1 ponto.
 - Implementação e uso de herança, polimorfismo e coleções: 1 ponto.
 - Tratamento de exceções: 1 ponto.
 - Persistência de dados em arquivos: 1 ponto.
 - Interface gráfica com o usuário: 1 ponto.
 - Implementação e execução do sistema conforme a descrição: 5 pontos.
- Ponto extra (opcional): será atribuído 1 (um) ponto extra, considerando o valor máximo total de 10 (dez) pontos na avaliação, se o sistema salvar e carregar os dados em arquivo-texto no formato JSON ou XML.

4. Entrega e apresentação:

- A entrega do Trabalho 2 envolverá:
 - códigos-fonte do sistema (e demais arquivos necessários para a compilação do sistema).
 - relatório do projeto.
- Deverá ser gerado um arquivo compactado (.zip ou .rar), com os itens acima, e entregue pelo Moodle da disciplina.
- A entrega deve ser realizada **individualmente**, mesmo que tenha sido desenvolvido em grupo, para confirmação do desenvolvimento do trabalho.
- A apresentação é obrigatória para a avaliação do funcionamento da aplicação.
 - A avaliação da execução será na apresentação.
 - Em caso de ausência na apresentação, a nota será de 50% do total.
- Data da entrega: **1º / 7 / 2026**.
- Datas de apresentação: **1º e 6 / 7 / 2026**.

5. Considerações finais:

- O trabalho pode ser desenvolvido individualmente, em dupla ou trio.
- A implementação deve seguir o Java Code Conventions para nomes de identificadores e estrutura das classes.
- Não serão aceitos trabalhos com erros de compilação. Programas que não compilarem corretamente terão nota zerada.
- A cópia parcial ou completa do trabalho terá como consequência a atribuição de nota 0 (zero) aos trabalhos dos alunos envolvidos. Para análise de similaridade será utilizado o MOSS (<https://theory.stanford.edu/~aiken/moss/>).

Apêndice: exemplos de arquivos de dados para o método **inicializar()** da classe ACMESpiele.

- Exemplo de arquivo de clientes: CLIENTESINICIAL.CSV. Formato de cada linha: número do cliente;nome do cliente;e-mail do cliente;tipo do cliente (1-individual, 2-corporativo);COF (se for individual) ou CNPJ (se for corporativo);nome fantasia (se for corporativo)

```
numero;nome;email;tipo;cpf_cnpj;nomeFantasia  
22;Coyote;coyote@toons.com;1;11111111-11  
4;ACME Corp.;acme@acme.com;2;33.333.333/0003-3;ACME Toons
```

- Exemplo de arquivo de tecnologias: JOGOSINICIAL.CSV. Formato de cada linha: código do jogo;nome do jogo;ano do jogo;valor diário do jogo;categoria

```
codigo;nome;ano;valorDiario;categoria  
444;War;2020;4.4;ESTRATEGIA  
333;Tetris;2000;3.3;AVENTURA
```

- Exemplo de arquivo de clientes: FORMASPAGAMENTOINICIAL.CSV. Formato de cada linha: código da forma de pagamento;dia do vencimento do pagamento;número do cliente; tipo da forma de pagamento (1-cartão de crédito, 2-PIX);número (se for cartão de crédito) ou chave (se for PIX);data de validade (formato DD/MM/AAAA - se for cartão de crédito)

```
cod;diaVencimento;numero;tipo;numero_chave;validade  
3333;3;4;1;33333333333333;3/3/2027  
1111;11;22;1;11111111111111;11/11/2026  
2222;22;4;2;acmepix@acme.com
```

- Exemplo de arquivo de vendas: CONTRATOSINICIAL.CSV. Formato de cada linha: identificador do contrato;data do contrato (formato DD/MM/AAAA);período (em dias);número do cliente;código do jogo;código da forma de pagamento

```
id;data;periodo;numero;codigo;cod  
33333;03/03/2026;33;22;444;2222  
44444;04/04/2026;44;4;333;3333
```